

Stack Definition:

A stack is a **linear data structure** where elements are stored in the **LIFO (Last In First Out)** principle where the last element inserted would be the first element to be deleted.

i.e., it is an ordered collection of items in which items may be inserted and deleted at one end, called the **top** of the stack.

Despite their restrictions, stacks have many uses. It is traditional to call the accessible element of the stack the **top** element. Elements are not said to be inserted, instead they are **pushed** onto the stack. When removed, an element is said to be **popped** from the stack.

Stack Representation

A stack allows all data operations at one end only. At any given time, we can only access the top element of a stack.

Stack can be represented in the memory as a linear array named "STACK" consisting of "N" elements or locations. It is possible for us to add and delete only one element at a time from the stack.

The process of adding an element to the stack represents an operation called as the 'PUSH' operation and the process of deleting an element from the stack represents an operation called as the 'POP' operation.

A variable named 'TOP' is used to contain the location of the topmost element of the stack.

If it is not possible for us to add another element to the stack because the stack is full it represents a situation called as the 'OVERFLOW' condition and similarly if it is not possible for to delete an element from the stack because the stack is empty it represents a situation called as the UNDERFLOW condition. The condition 'TOP=-1' will indicate the stack is empty.

STACK OPERATIONS

The different operations possible on stack are:

1. Push: Adds a value to the top of the stack. The size of the stack will increase by one.
2. Pop: Deletes and returns the value at the top of the stack. The size of the stack will decrease by one.
3. Size: Obtains the number of elements in the stack.
4. Peek: Returns a value but does not delete the value at the top of the stack.
5. Underflow: When there are no elements on the stack.
6. Overflow: When the stack is full.

PUSH Operation

The process of adding one element or item to the stack is represented by an operation called as the PUSH operation. The new element is added at the topmost position of the stack. However, before inserting the value, we must first check whether space is available to add the element and then add the element at the topmost position in the stack.

ALGORITHM: PUSH(STACK, TOP, ITEM) This algorithm inserts an element ITEM to the top of the stack represented by an array STACK containing N elements with a pointer TOP denoting the top element in the stack.

```
Step 1 :[ Check for overflow ]
        If TOP = N-1 Then
            Print 'Stack is full or Overflow'
            Exit
        [ EndIf ]
Step 2 :    [ Increment TOP pointer ]
            TOP = TOP + 1
Step 3 :    [ Insert an ITEM ]
            STACK[TOP] = ITEM
Step 4 :    Return
```

The C implementation of above algorithm is as shown in the following function:

```
void PUSH(int item )
{
    if(TOP == MAX-1)           // Check for stack overflow
    {
        printf("Stack Overflow\n");
        exit(0);
    }
    else
    {
        TOP++;                // Increment TOP value by 1
        STACK[TOP]=item;
    }
}
```

The above function adds a new item to the STACK. Before inserting the item, the PUSH() function checks the condition TOP==MAX-1. If this condition is true the STACK is full and no further insertions can be done on it. If an attempt is made to insert a value in a STACK that is already full, a message is displayed that there is an overflow and we exit from the function. If the condition is false, TOP value is incremented by 1 and store the new item at the position given by the STACK[TOP].

POP Operation

The process of deleting one element or item from the stack is represented by an operation called as the POP operation. Here we will have to check whether there are any elements for us to delete if so we will then have to delete the topmost element from the stack.

ALGORITHM: POP(STACK, TOP, ITEM) This algorithm removes the top element from a stack which is represented by a array STACK and return this element. TOP is a pointer to the top of the element.

```
Step 1 :[ Check for Underflow ]
        If TOP = -1 Then
```

```

                                Print 'Stack is empty or Underflow'
                                Exit
                                [ EndIf ]
Step 2 :    [ Return top element of the stack ]
                                ITEM = STACK[TOP]
Step 3 :    [ Decrement pointer ]
                                TOP = TOP - 1
Step 4 :    Return (ITEM)

```

The C implementation of above algorithm is as shown in the following function:

```

int POP()
{
    int item;
    if(TOP== -1)           //      check for empty stack
    {
        printf("Stack Underflow\n");
        exit(0);
    }
    else
    {
        item = STACK[TOP];    //      remove topmost element
        TOP--;                //      Decrement TOP pointer
        return item;         //      return topmost element
    }
}

```

The above function is used to delete topmost element from the STACK. However, before deleting the value, we must first check if TOP== -1, as it indicates that the STACK is empty and therefore no further deletions can be done on it. If an attempt is made to delete an item from the STACK that is already empty. If the condition is false, TOP value is decremented by 1 so that it points to the preceding element. The POP function deletes the item from the STACK and the item is returned.

The following program illustrates all the STACK operations.
Program to illustrate STACK operations (PUSH and POP operations).

```

#include<stdio.h>
#include<conio.h>
#define MAX 5

void PUSH(int); // function prototype to push an item
int POP();      // function prototype to pop an item
void DISPLAY(); // function prototype to display stack items
static int TOP=-1;
int STACK[MAX];
/*      Main Function      */
void main()

```

```

{
int choice, item;
while(1)
{
    clrscr();
    Status=1;
    printf("Stack Operations:\n");
    printf(" 1. Push \n");
    printf(" 2. Pop \n");
    printf(" 3. Display Contents \n");
    printf(" 4. Exit \n");
    printf("Enter your choice: \n");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1: printf("Enter item to Push: \n");
                scanf("%d",&item);
                PUSH(item);
                break;
        case 2 : item=POP();
                printf("Poped item is : %d\n",item);
                break;
        case 3 : printf("The Stack contents is:\n");
                DISPLAY();
                break;
        default : printf("Invalid choice!!!!!! \n");
                getch();
    }
}
}

/*      Function to push an item onto a Stack      */
void PUSH(int item )
{
    if(TOP == MAX-1)           //      Check for stack overflow
    {
        printf("Stack Overflow\n");
        exit(0);
    }
    else
    {
        TOP++;           // Increment TOP value by 1
        STACK[TOP]=item;
    }
}

/*      Function to pop an item from the Stack      */
int POP()
{

```

```

int item;
if(TOP== -1)          //      check for empty stack
{
    printf("Stack Underflow\n");
    exit(0);
}
else
{
    item = STACK[TOP]; //      remove topmost element
    TOP--;             //      Decrement TOP pointer
    return item;       //      return topmost element
}
}

//      Function to Display Stack elements
void DISPLAY()
{
    inti;
    if(TOP== -1)
        printf("Empty stack!!!!!!! \n");
    else
        for(i=TOP; i>=0; i--)
            printf("STACK[%d]= %d \n ",i,STACK[i]);
    getch();
}

```

APPLICATIONS OF STACK

There are several applications and uses of stacks:

1. Polish notation
2. Evaluation of Postfix Expression
3. Checking for Balanced brackets or Parentheses matching
4. Reversing a string
5. Recursion

POLISH NOTATION

An expression is a combination of operators and operands. In the standard method we are used to writing expressions containing binary operator as:

<Operand1>< Operator><Operand2>

Example:

A + B

This way of writing an expression, i.e., A+B is called **infix notation**. The expression is called *infix expression*. In infix expression the operation (here operation is +) comes between the two operands. While evaluating infix expressions we are faced with the situation of scanning through the expression many times, because an arithmetic expression cannot be executed from left to right. We will have to apply the rules of precedence while evaluating the expression.

Thus we realize that evaluation of an infix expression is a tedious task. Is it possible for us to have the expression in some other form so that it can be evaluated easily? The answer is ‘yes’. We can convert the expression into ‘*polish notation*’. The process of writing the operators of an expression either before their operands or after them is called the **Polish notation**, named after the Polish mathematician **Jan Lukasiewicz**.

In prefix notation, the operator comes before operands.

Example: $A + B$ whereas in *postfix* notation operator comes after operands.

Example: A B +

Example: Convert the infix expression $(A + B) * (C - D) * E$ into prefix expression.

1. a. Convert the sub-expression $(A+B)$ into prefix form i.e., it now becomes $+ A B$.
 b. Let us now replace $+A B$ with X .
2. a. Convert the sub-expression $(C-D)$ into prefix form i.e., it now becomes $- C D$.
 b. Let us now replace $- C D$ with Y .
3. The resulting partial infix expression is: $X * Y * E$.
4. So prefix expression for the obtained partial infix expression is: $** X Y E$.
5. The final prefix expression after replacing prefix expression for X and Y will be:

$$** + A B - C D E$$

Example 4.5: Convert the infix expression $(A + B) * (C - D) * E$ into postfix expression.

1.
 - a. Convert the sub-expression $(A + B)$ into postfix form i.e., it now becomes **A B +**.
 - b. Let us now replace **A B +** with X.
2.
 - a. Convert the sub-expression $(C - D)$ into postfix form i.e., it now becomes **C D -**.

- b. Let us now replace $C D -$ with Y .
3. The resulting partial infix expression is: $X * Y * E$
4. So postfix expression for the obtained partial infix expression is: $X Y * E *$
5. The final postfix expression after replacing postfix expression for X and Y will be:

$$A B + C D - * E *$$

EVALUATION OF POSTFIX EXPRESSION

When we use an infix expression it is very difficult for the computer to evaluate it. The evaluation of a postfix expression is much easier. There are certain points for us to understand, such as the postfix expression always does not contain any parenthesis. The following algorithm can be used to evaluate the postfix expression.

ALGORITHM : EVALUATION-OF-POSTFIX

- Step 1:** Input the postfix expression.
- Step 2:** Scan the post fix expression from left to right one character at a time.
- Step 3:** If the character is an operand put it into the STACK
- Step 4:** If the character is an operator then:
- a. Pop two elements from the STACK
 - b. Evaluate the expression based on the operands and operator
 - c. Place the result back into the STACK.
- Step 5:** Repeat Steps 2, 3 and 4 for all the characters in the postfix expression.
- Step 6:** Pop the final result from the stack.
- Step 7:** Exit

Example: Let us consider another example for the evaluation of a postfix expression. Consider the expression $A B + C D E - * +$

PostFix Expression	Character	Value	Stack
$A B + C D E - * +$			
$B + C D E - * +$	A	Value of A = 5 PUSH (5)	5
$+ C D E - * +$	B	Value of B = 3 PUSH (3)	3,5
$C D E - * +$	+	POP two values $n_2=3$ $n_1=5$, RESULT=5+3=8, PUSH (8)	8
$D E - * +$	C	Value of C = 5 PUSH (5)	5,8
$E - * +$	D	Value of D=7 PUSH (7)	7,5,8
$- * +$	E	Value of E=2 PUSH (2)	2,7,5,8
$* +$	-	POP two values $n_2=2$ $n_1=7$, RESULT=7-2=5, PUSH (5)	5,5,8

+	*	POP two values $n_2 = 5$, $n_1 = 5$, RESULT=5*5=25,PUSH(25)	25,8
	+	POP two values $n_2=25$, $n_1=8$, RESULT= 8+25=33 ,PUSH(33)	33

Since all the characters of the postfix expression is complete, pop the final result from the stack,
So **Final Result** is 33

TRANSLATION FROM INFIX TO POSTFIX

There are number of compilers that convert infix expressions into Postfix expressions. In such compilers invalid Postfix expressions, and consequently their invalid infix expression counterparts, must be detected in the compiling phase of translation. The following section will describe a method that can be used to detect such invalid expressions.

We have to devise an algorithm which will convert an infix expression into Postfix expression. This conversion is based on the precedence of the operators and requires the use of stack. Let us first assign precedence values to the five operators. The precedence associated with exponentiation (^) is greater than that of multiplication and division. The precedence associated with multiplication and division is greater than the addition and subtraction.

Operators	Priority
^	3
* /	2
+ -	1
#	0

Assume that the stack has been initialized to a special symbol “#” which has a precedence value less than all other precedence values given in the table. A general algorithm to convert infix into postfix is given as follows:

Conversion from infix to postfix expression:

Suppose “I” is an infix expression and we have to find the equivalent postfix expression “P”.

- Step 1:** Initialize the stack.
- Step 2:** Scan the Infix expression left to right reading one character at a time.
- Step 3:** If the character is an operand add it to the Postfix expression **P**.
- Step 4:** If the character is a left parenthesis push it to **STACK**.
- Step 5:** If the character is an operator **X** then
 - a. Repeatedly pop one operator from the **STACK** and add it to **P** each operator which has the same precedence as or higher precedence than **X**.
 - b. Add **X** to **STACK**.
- Step 6:** If the character is a right parenthesis then
 - a. Repeatedly **POP** from **STACK** and add to **P** each operator until a left parentheses is encountered.

- b. Remove the left parentheses from stack
[Do not add it to P]

Step 7: Repeat Steps 2,3,4,5 and 6 till all the characters are read in the Infix expression.

Step 8: Exit

Example: Consider the following expression:

$$A + B * C$$

The contents of the postfix string, and STACK are shown after scanning each symbol. STACK is shown with its top to the right.

Step	Character	STACK	Postfix	Remarks
		#		Initialize the stack with the '#' symbol
1	A		A	The Character A is an operand, Add to Postfix expression.
2	+	#,+	A	The Character(+) is an operator whose precedence is greater than Stack operator, So Push the symbol to the stack.
3	B	#,+	AB	The Character B is an operand, Add to Postfix expression.
4	*	#,+,*	AB	The Character(*) is an operator whose precedence is greater than Stack operator, So Push the symbol to the stack.
5	C	#,+,*	ABC	The Character C is an operand, Add to Postfix expression.
6		#,+	ABC*	Pop one operator(*) from stack and add to Postfix expression.
7		#	ABC*+	Pop one operator(+) from stack and add to Postfix expression.

The Equivalent Postfix expression is ABC*+

Program to convert Infix expression to Postfix expression

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#include<conio.h>
#include<stdlib.h>
```

```

/*    Global declaration */
char  STACK[50];
int   TOP=-1;

/*    Function to push an item onto Stack    */
void  PUSH(char ele)
{
    STACK[++TOP]=ele;
}

/*    Function to remove element from the stack    */
char POP()
{
    return(STACK[TOP--]);
}

/*    Function to assign priority to operators    */
int P(char ch)
{
    if ((ch=='') || (ch=='('))
        return(0);
    else if ((ch=='+') || (ch=='-'))
        return(1);
    else if ((ch=='*') || (ch=='/'))
        return(2);
    else    return(-1);
}

/*    Main Program    */
void main()
{
    char  infix[50],postfix[50];
    char  ch,sch;
    int   i,j=0;

    clrscr();
    printf("Enter an infix expression\n");
    gets(infix);

    PUSH('(');

    for(i=0; i<strlen(infix); i++)
    {
        ch = infix[i];

        if((((ch>='A')&&(ch<='Z')) || ((ch>='a')&&(ch<='z'))))

```

```

        postfix[j++]=ch;
    else if((ch>='0')&&(ch<='9'))
        postfix[j++]=ch;
    else
    {
        if(P(ch)==-1)
        {
            printf("Wrong Operator used\n");
            exit(0);
        }
        if(ch=='(')
            PUSH(ch);
        else if (ch==')')
        {
            while(STACK[TOP]!='(')
                postfix[j++]=POP();
            POP();
        }
        else
        {
            sch=POP();
            while(P(sch)>P(ch))
            {
                postfix[j++]=sch;
                sch=POP();
            }
            PUSH(sch);
            PUSH(ch);
        }
    }
}
while(STACK[TOP]!='')
    postfix[j++]=POP();

postfix[j]='\0';

printf("The Infix expression is %s\n",infix);
printf("The Postfix expression is %s\n",postfix);
getch();
}

```

Program to evaluation of Postfix expression

```

#include<stdio.h>
#include<ctype.h>

int    STACK[100];
int    TOP = -1 ;

```

```

void PUSH(int item)
{
    STACK[++TOP]= item;
}

int POP()
{
    return(STACK[TOP--]);
}

void EvalPostfix(char postfix[])
{
    int    i,val,A,B ;
    char   ch;

    for (i = 0 ; postfix[i] != '\0'; i++)
    {
        ch = postfix[i];
        if (isdigit(ch))
            PUSH(ch - '0');
        else if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
        {
            A = POP();
            B = POP();

            switch (ch) /* ch is an operator */
            {
                case '*':
                    val = B * A;
                    break;

                case '/':
                    val = B / A;
                    break;

                case '+':
                    val = B + A;
                    break;

                case '-':
                    val = B - A;
                    break;
            }
            PUSH(val);
        }
    }
    printf( " \n Result of expression evaluation : %d \n", POP() );
}

```

```

void main()
{
    inti ;

    /* declare character array to store postfix expression */
    char postfix[POSTFIXSIZE];

    printf( " \nEnter postfix expression : \n ");

    for (i = 0 ; i<= POSTFIXSIZE - 1 ; i++)
    {
        scanf("%c", &postfix[i]);
        if ( postfix[i] == ')' )
            break;
    }

    EvalPostfix(postfix);
}

```

QUEUE INTRODUCTION

A queue is a Linear data structure which follows the principle of “First-in-First-out” (FIFO) i.e., it is an ordered collection of items in which items may be inserted at one end and deleted at another end.

Some more examples of requirements for a queue are, generally, a user can request that one item be printed and then request an additional document before the first is completed. The list of items waiting to be printed is a queue. If you rapidly click the mouse twice, once on each of two “buttons,” and the first button requires some time to respond, the second click is kept in a queue until it can be processed.

REPRESENTATION OF A QUEUE USING AN ARRAY

Queue can be represented in the memory as a linear array named “Q” consisting of “N” elements or locations. It is possible for us to add and delete only one element at a time from the queue. Two variables named **FRONT** and **REAR** are used to identify the locations of the first and the last elements in the queue. The insertion of a new element can be done only at the **rear** end of the queue and the deletion of an element can be done only at the **front** end of the queue. Another important difference between a natural queue and a queue data structure is that when an element is deleted in the array queue the other elements are not rearranged.

If it is not possible for us to add another element to the queue because the queue is full it represents a situation called as the **OVERFLOW** condition. The condition **REAR=N-1** will indicate that the queue is full. Similarly if it is not possible for to delete an element from the queue because the queue is empty it represents a situation called as the **UNDERFLOW** condition. The condition **FRONT=-1** will indicate the queue is empty.

QUEUE OPERATIONS

The different operations can be applied to a queue are given below:

1. **QInsert (Enqueue):** Adds a value to the rear of the queue. The size of the queue will increase by one.
2. **QDelete (Dequeue):** Deletes and returns the value at the front of the queue. The size of the queue will decrease by one.
3. **Size:** Obtains the number of elements in the queue.
4. **Underflow:** When there are no elements on the queue.
5. **Overflow:** When the queue is full.

QUEUE INSERTION OPERATION (ENQUEUE)

The process of adding one element or item to the queue is represented by an operation called as the Insertion operation. Here we will have to first check whether space is available for us to add the element and then add the element at the rear position in the queue.

ALGORITHM: QINSERT(Q, FRONT, REAR, ITEM) Let **Q** be a Queue consisting of **N** elements. **FRONT** and **REAR** are positions of the front and rear elements of a queue. This algorithm inserts **ITEM** at the rear end of the queue.

```
Step 1: [Check for overflow]
        If REAR = N - 1 Then
            Print "Queue is full or Overflow"
            Exit
        [EndIf]
Step 2 : [Reposition the FRONT and REAR pointers]
        If FRONT = -1 Then [Insert ITEM as first element of the queue]
            FRONT = 0
            REAR = 0
        Else
            REAR = REAR + 1    [Increment REAR pointer]
        [EndIf]
Step 3 : Q[REAR] = ITEM
Step 4 : Return
```

The Cimplementation of above algorithm is as shown in the following function:

```
/*    Function to Add an item onto a Queue    */
void QINSERT( int item )
{
    if(REAR==N-1)
    {
        printf("Queue Overflow\n");
        exit(0);
    }
}
```

```

else
{
    REAR++;          // increment REAR by 1
    Q[REAR]=item;
    if ( FRONT== -1)
        FRONT = 0;  // Set FRONT to 0
}
}

```

The above function adds a new item to the queue Q. Before inserting the item, the QINSERT() function checks the condition REAR==MAX-1, where MAX is the size of the queue Q. If this condition is TRUE the queue Q is full and no further insertions can be done on it. If an attempt is made to insert a value in a queue Q that is already full, an OVERFLOW message is printed. If the condition is FALSE, the REAR value is incremented by 1. If the queue Q is empty the FRONT value is set to 0. Then the new item is inserted at the position given by the Q[REAR].

QUEUE DELETION OPERATION (DEQUEUE)

The process of deleting one element or item from the queue is represented by an operation called as the **deletion** operation. Here we will have to check whether there are any elements for us to delete if so we will then have to delete the Front most element from the queue.

ALGORITHM: QDELETE(Q, FRONT, REAR, ITEM) Let Q be a Queue consisting of N elements. FRONT and REAR are positions of the front and rear elements of a queue. This algorithm deletes the front element of the queue and returns the element.

```

Step 1: [ Check for Underflow ]
           If FRONT = -1 Then
               Print "Queue is empty or Underflow"
               Exit
           [EndIf]
Step 2: [ Delete front element ]
           ITEM = Q[FRONT]
Step 3: [Check whether deleted element is the last element]
           If FRONT = REAR Then      [Queue has only one element]
               FRONT = -1
               REAR = -1
           Else                      [Increment FRONT pointer]
               FRONT = FRONT + 1
           [EndIf]
Step 4: Return (ITEM)

```

The Cimplementation of above algorithm is as shown in the following function:

```

/*      Function to Remove an item onto a Queue    */

```

```

int QDELETE()
{
    int item;
    if(FRONT == -1)
    {
        printf("Queue Underflow\n");
        exit(0);
    }
    else
    {
        ITEM = Q[FRONT];
        if(FRONT == REAR) //Queue has only one element
        {
            FRONT = -1;
            REAR = -1;
        }
        else // Increment FRONT pointer
            FRONT = FRONT + 1;
        return (ITEM);
    }
}

```

Program: *Program to illustrate QUEUE operations.*

```

#include<stdio.h>
#include<conio.h>
#define MAX 5

void QINSERT(int); // function prototype to add an item
int QDELETE(); // function prototype to delete an item
void DISPLAY(); // function prototype to display items
static int FRONT = -1, REAR = -1;
int Q[MAX];
/* Main Function */
void main()
{
    int choice, item;
    while(1)
    {
        clrscr();
        printf("Queue Operations:\n");
        printf("1. ADD \n");
        printf("2. DELETE \n");
        printf("3. DISPLAY Contents \n");
        printf("4. Exit \n");
        printf("Enter your choice: \n");
        scanf("%d",&choice);
    }
}

```

```

        switch(choice)
        {
            case 1: printf("Enter item to ADD: \n");
                     scanf("%d",&item);
                     QINSERT(item);
                     break;
            case 2 : item=QDELETE();
                     printf("Deleted item is : %d\n",item);
                     break;
            case 3 :   printf("The QUEUE contents is:\n");
                     DISPLAY();
                     break;
            default :   printf("Invalid choice!!!!!! \n");
                       getch();
        }
    }
}

```

```

/*      Function to Add an item onto a Queue   */
void QINSERT( int item )
{
    if(REAR==N-1)
    {
        printf("Queue Overflow\n");
        exit(0);
    }
    else
    {
        REAR++;           //    increment REAR by 1
        Q[REAR]=item;
        if ( FRONT==-1)
            FRONT = 0;    // Set FRONT to 0
    }
}

```

```

/*      Function to Remove an item onto a Queue   */
int QDELETE()
{
    int item;
    if(FRONT==-1)
    {
        printf("Queue Underflow\n");
        exit(0);
    }
    else
    {
        ITEM = Q[FRONT];
    }
}

```

```

        if(FRONT == REAR) //Queue has only one element
        {
            FRONT = -1;
            REAR = -1;
        }
        else // Increment FRONT pointer
            FRONT = FRONT + 1;
        return (ITEM);
    }
}

void DISPLAY()
{
    inti;
    if(FRONT==-1)
        printf("Empty Queue!!!!!!! \n");
    else
        for(i=FRONT; i<=REAR; i++)
            printf("Q[%d]= %d \n ",i,Q[i]);
    getch();
}

```

CIRCULAR QUEUE

Unlike a natural queue where the elements are shuffled when an element is deleted in a linear queue we do not shuffle or rearrange the elements. This situation creates certain problems when we have the REAR variable pointing to the last data and we have a number of positions empty at the starting of the queue.

The solution to this problem is not to have the program stop when the limit of the array used to store the queue is reached, but we can have both the REAR variable and FRONT variable loop back to the array. In this way, any number of items could be placed on the queue, so long as items were also being taken off. This implementation of a queue is called *circular queue* because it uses its storage array as if it were a circle instead of a linear list.

Thus a Circular queue is visualization where the elements of a linear queue are visualized to be arranged in a circle following the rule of First-in-First-out (FIFO). Conceptually, the array used for the circular version of the program looks like Figure 4.13. Thus the following points can be used to describe a circular queue.

- a. Circular queue is a linear data structure. It follows the principle of FIFO.
- b. In circular queue the last node or location is connected back to the first node or location to make a circle.
- c. Elements are added at the rear end and elements are deleted at front end of the queue.
- d. Both the FRONT and the REAR index points to the beginning of the array at the start.
- e. The queue is full only when the REAR index is one less than FRONT otherwise, there is a room in the queue for another element to be added.

ALGORITHM FOR INSERTION AN ITEM IN A CIRCULAR QUEUE

ALGORITHM: CIRCULAR_QUEUE_INSERTION :CQ is a circular queue consisting of N elements. *FRONT* and *REAR* are pointers to the front and rear elements of a queue. This algorithm inserts ITEM at the rear end of the queue.

```
Step 1: [ Check for overflow ]
        If (FRONT = 1 and REAR = N) OR (FRONT=REAR + 1) Then
            Display "CQ Overflow!"
            Exit
        [EndIf]
Step 2 : [ Check for empty queue ]
        If FRONT = -1 Then
            FRONT=0 and REAR=0      [Insert ITEM as first element of the queue ]
        Else If REAR = N-1 Then      [ Reset the REAR pointer ]
            REAR =0
        Else                          [ Increment REAR pointer ]
            REAR = REAR + 1
        [EndIf]
Step 3 :CQ[REAR] ← ITEM
Step 4 : Return
```

ALGORITHM TO DELETE AN ITEM FROM A CIRCULAR QUEUE

ALGORITHM: CIRCULAR_QUEUE_DELETION:CQ is a circular queue consisting of N elements. *FRONT* and *REAR* are pointers to the front and rear elements of a queue. This algorithm deletes front element of the queue.

```
Step 1: [ Check for Underflow ]
        If FRONT = -1 Then
            Display "Underflow!"
            Exit
        [EndIf]
Step 2 : [ Delete front element ]
        ITEM = Q[FRONT]
Step 3 : [Check Empty queue ]
        If FRONT = REAR Then          [Queue has only one element ]
            FRONT=-1 and REAR =-1
        Else If FRONT = N-1 Then      [ Reset the FRONT pointer ]
            FRONT =0
        Else                          [ Increment FRONT pointer ]
            FRONT =FRONT + 1
        [EndIf]
Step 4 : Return
```

Program:*Program to perform insertion and deletion operations on circular queue.*

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#define MAX 5

void CQINSERT(int); // function prototype to add an item
int CQDELETE();     // function prototype to delete an item
void DISPLAY();     // function prototype to display items
static int FRONT=-1, REAR=-1;
int CQ[MAX];

/*    Main function    */
void main()
{
    int choice, item;
    while(1)
    {
        clrscr();
        printf("Circular Queue Operations:\n");
        printf(" 1. ADD \n");
        printf(" 2. DELETE \n");
        printf(" 3. DISPLAY Contents \n");
        printf(" 4. Exit \n");
        printf("Enter your choice: \n");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1: printf("Enter item to ADD: \n");
                     scanf("%d",&item);
                     CQINSERT(item);
                     break;
            case 2 : item=CQDELETE();
                     printf("Deleted item is : %d\n",item);
                     break;
            case 3 : printf("The CIRCULAR QUEUE contents is:\n");
                     DISPLAY();
                     break;
            default : printf("Invalid choice!!!!!! \n");
                     getch();
        }
    }
}

/*    Insert an item in a circular queue    */
void CQINSERT(int item)
{

```

```

if((FRONT==0 && REAR==MAX-1) || (FRONT==(REAR)+1))
{
    printf("Queue is full. Overflow!");
    exit(0);
}
else
{
    if(FRONT== -1)        /*    Empty queue    */
    {
        FRONT=0;
        REAR=0;
    }
    else if(REAR==MAX-1)
        REAR=0;
    else
        REAR++;
    Q[REAR]=item;
}
}
/*    Delete an item from a circular queue    */
intCQDELETE()
{
    int item;
    if(FRONT== -1)        /*    Empty queue    */
    {
        printf("Queue is empty or Underflow!");
        exit(0);
    }
    else
    {
        item=Q[FRONT];
        if(FRONT==REAR)
        {
            FRONT=-1;
            REAR=-1;
        }
        else if(FRONT==MAX-1)
            FRONT=0;
        else
            FRONT++;
        return(item);
    }
}
/*    Function to display circular queue elements    */
void DISPLAY()
{
    inti;

```

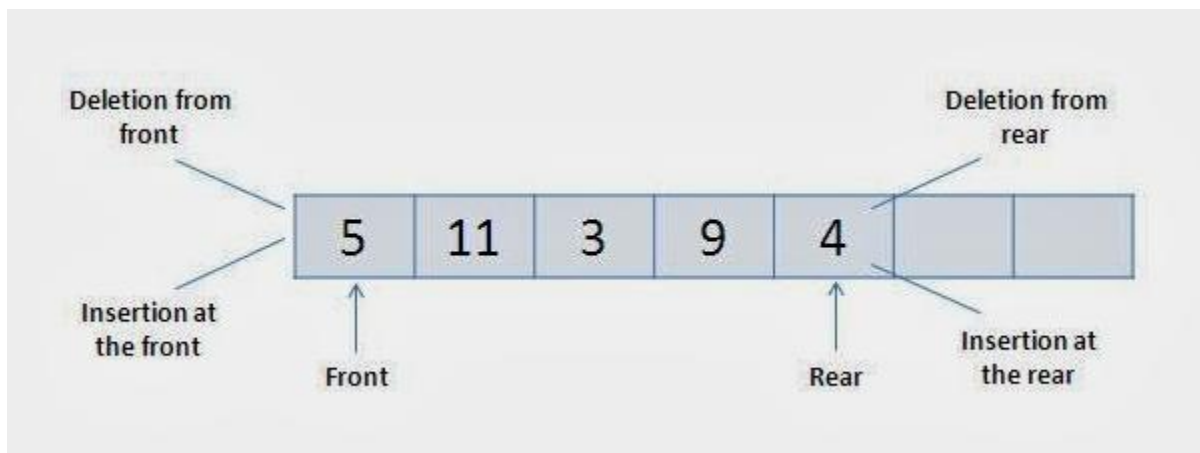
```

if(FRONT==-1)
    printf("Empty Queue!!!!!!! \n");
else
    for(i=FRONT; i<=REAR; i++)
    {
        printf("Q[%d]= %d \n ",i,Q[i]);
        if(i==MAX-1) i=0;
    }
    getch();
}

```

DOUBLE ENDED QUEUE

Consider a queue-like data structure that supports insertion and deletion at both the front and rear of the queue. Such an extension of a queue is called a *double-ended queue* or a *deque*. A deque is a linear list in which elements can be added or removed at either end but not in the middle.



Double ended Queue

There are two variations of deque namely, an *input-restricted* deque and an *output restricted* deque which are intermediate between a deque and a queue. Specifically, an input-restricted deque is a deque which allows insertions at only one end of the list but allows deletions at both ends of the list and similarly an output-restricted deque is a deque which allows deletions at only one end of the list but allows insertion at both ends of the list.

PRIORITY QUEUES

Priority Queue is an ordered list of homogeneous elements. In normal queue, service is provided on the basis of First-In-First-Out. In a priority queue service isn't provided on the

basis of First-In-First-Out service, but rather then each element has a priority based on the urgency of the need.

An element with higher priority is processed before other elements with lower priority. Elements with the same priority are processed on First-In-First-Out service basis. An example of priority queue is a hospital waiting room. A patient having a more fatal problem would be admitted before other patients.

Other applications of priority queues are found in long term scheduling of jobs processed in a computer. In practice, short processes are given a priority over long processes as it improves the average response of the system.

APPLICATIONS OF QUEUES

1. In a computer network messages from one computer to another computer are generally created synchronously. These messages need to be buffered until the receiving computer is ready for it. These communication buffers make use of queues by storing these messages in a queue. Also the messages need to be sent to receiving computer in the same order in which they are created.
2. Queues are used in timesharing system in which programs form a queue while waiting to be executed.